Group 19

Java O-O Program Visualizer/Animator

Requirement Document

Names:
Prottasha Deb
Angelina Martis
Arya Patel
Aiden Stensrud

Section:
CSE 4101 Section 1

Date: 09/28/2026

TABLE OF CONTENTS

1. DOCUMENT PURPOSE **3**

2. PROJECT SCOPE **3**

1. Educational Problem 3
2. Scope of the Application 3
3. Scope decisions still required 3

3. DEFINITIONS AND ACRONYMS **4**

4. REFERENCES **4**

5. OVERVIEW **4**

1. Product Perspective: 4
2. Product Functions: 5
3. User characteristics: 5
4. Typical Interaction 5

6. CONSTRAINTS, ASSUMPTIONS, AND DEPENDENCIES **6**

1. Deployment and Implementation Constraints 6
2. Assumptions About Use 6
3. Dependencies 6
4. Decisions to be Resolved 6
5. Managing Changes 7

7. APPORTIONING OF REQUIREMENTS **7**

Milestone 1 7

Milestone 2 7

Milestone 3 7

8. FUNCTIONAL & INTERFACE REQUIREMENTS **8**

1. Enter and Load Source 8
2. Analyze Supported Source 8
3. Generate Execution States 8
4. Navigate Forward 8
5. Synchronize Code and Diagram 8
6. Display the Memory Model 9

1. DOCUMENT PURPOSE

This Software Requirements Specification (SRS) describes what the Java Object-Oriented Program Visualizer/Animator is expected to do and how its behavior can be checked. It guides the project team through design, implementation, demonstrations, and testing across the three project milestones. It also gives instructors and other reviewers a consistent description of the intended product.

Requirements describe observable application behavior. Design documentation will explain how that behavior is implemented, including the parser representation, storage of execution states, and rendering technology. Keeping these responsibilities distinct allows implementation choices to change without altering the application's educational purpose.

2. PROJECT SCOPE

1. Educational Problem

Students in introductory courses such as CSE 1001 and CSE 1002 may be able to read Java statements, but not understand how they affect the program state. Common difficulties include distinguishing classes from objects, following changes to primitive values, understanding references, and identifying when local variables are available during a method call.

The visualizer addresses these difficulties by integrating source code and memory diagram representations into a single workflow. Students can examine one execution state at a time and connect the highlighted source location with the corresponding change in the diagram. Instructors can use the same representation to explain examples during lectures or guided exercises.

2. Scope of the Application

The product is a desktop application that processes Java source locally using a compiler pipeline developed by the project team. The pipeline analyzes source code and supplies the execution information needed by the memory diagram generator and animator. The intended workflow includes entering or loading code, analyzing it, and moving forward or backward through its execution states.

The visualization represents an educational model of execution rather than physical memory addresses or the internal layout of a particular Java Virtual Machine. It must clearly distinguish primitive values, reference variables, objects, strings, and active method frames to support introductory explanations.

3. Scope decisions still required

The exact subset of the Java language that is supported remains to be determined. A compiler built for this project must not be assumed to support every Java feature or standard library. We must document supported constructs before treating a program as a valid acceptance example. Source-size limits, execution limits, supported operating systems, and minimum hardware specifications also remain open decisions.

3. DEFINITIONS AND ACRONYMS

- CSE 1001 and CSE 1002: Introductory computer science courses that new Computer Science students take
- Lexical analyzer: A compiler component that groups source characters into tokens such as identifiers, keywords, operators, and literals.
- Parser: A compiler component that checks token sequences against a grammar and builds a structured representation of the program.
- State interpreter: The component that evaluates the supported program representation and produces execution states for visualization.
- Execution state: The values, active method frames, objects, and reference relationships present at a given step in execution.
- Stack frame: The diagram region associated with an active method invocation, including the parameters and local variables.
- Memory diagram: An educational display of program state, including frames, primitive values, objects, and references.
- Reference and reference arrow: A reference identifies an object or is null. An arrow depicts a non-null relationship from a reference variable to its target object, which may be a string.
- Execution trace: An ordered sequence of execution states and associated source locations used for forward and backward navigation.

4. REFERENCES

The following references were used as background reading on memory diagrams.

[1] Holliday, Luginbuhl, “A Guide to Using Memory Diagrams,” Western Carolina University.
https://agora.cs.wcu.edu/~holliday/teaching/LectureNotes/150/MemoryDiagramGuide.pdf

[2] Holliday, Luginbuhl, “Using Memory Diagrams When Teaching a Java-Based CS1,” Proceedings of the 41st Annual ACM Southeast Conference.
https://www.researchgate.net/profile/Mark-Holliday-2/publication/242406239_Using_Memory_Diagrams_When_Teaching_a_Java-Based_CS1/links/0c96052b8ef4e9e9b7000000/Using-Memory-Diagrams-When-Teaching-a-Java-Based-CS1.pdf

[3] Holliday, Luginbuhl, “CS1 Assessment Using Memory Diagrams,” Proceedings of 35th SIGCSE Tech Symposium on CS Education.
https://dl.acm.org/doi/pdf/10.1145/971300.971373

5. OVERVIEW

1. **Product Perspective:**

   The application operates on the user’s machine and does not depend on a cloud compiler for source analysis or visualization. Its main parts are the code editor, compiler,

state interpreter, memory diagram generator, and animation controls. These parts must use a consistent view of the source program and the currently selected execution state.

The compiler pipeline produces structured information about the supported program. The state interpreter uses that information to describe execution, and the renderer translates each state into a diagram. This separation allows the team to test program behavior independently of layout and animation.

2. Product Functions:

Code editing allows the user to enter, modify, clear, or load Java source. The editor displays line numbers and highlights the source location associated with the selected execution step. Source text is the input to analysis; changing that text requires a new analysis before the resulting trace can be treated as current.

Program analysis includes lexical analysis and parsing, followed by the interpretation needed to obtain execution states. Memory rendering displays primitive values in labeled rectangles, reference variables in labeled boxes, and strings in rounded or oval nodes. Reference arrows connect variables to their represented objects.

Interactive navigation allows the user to inspect the execution sequence in both directions. The diagram and source highlighting change together, so that the displayed line and displayed state refer to the same selected step. Animation communicates the transition between states without changing the values or relationships being represented.

3. User characteristics:

Students are expected to have basic computer skills but may have little programming experience. Labels and controls should use familiar language and should make the distinction between a value and a reference visible. The interface should support deliberate inspection rather than require a student to follow a continuously changing display.

Instructors use the application to demonstrate Java concepts and discuss program behavior. They need consistent diagram conventions so that examples shown in class match the meanings students learn while working independently. Both groups use the same underlying execution model.

4. Typical Interaction

The user enters or loads a supported Java program and requests analysis. If analysis succeeds, the application makes its execution states available for inspection. The user advances to observe changes and compares the diagram with the highlighted code. After editing the source, the user analyzes the revised program to begin a new trace. When analysis fails, the interface provides information that helps the user correct the input.

6. CONSTRAINTS, ASSUMPTIONS, AND DEPENDENCIES

1. Deployment and Implementation Constraints

The delivery target is a desktop application. Earlier milestone research compared web and desktop technologies and led to the finalization of the desktop scope. Source analysis and diagram generation are intended to run locally without a cloud compilation service.

The project includes a compiler pipeline developed by the team. The supported Java subset must be stated explicitly so that implementation effort and demonstrations remain consistent with the project schedule. The selected user interface and graphics technologies must be able to render labeled nodes and reference arrows and update them during navigation.

2. Assumptions About Use

Users are assumed to be able to launch the application, enter text, open a source file, and activate interface controls. Examples are expected to support introductory programming instruction. A user should not need compiler expertise to interpret the basic output, although the Java concepts illustrated by an example may require classroom explanation.

The host computer is assumed to meet the eventual platform and hardware requirements. The phrase “modern machine” alone is not a testable specification; we must identify the supported operating systems, required runtime, memory needs, and relevant display requirements before final deployment testing.

3. Dependencies

The application depends on a compatible host operating system, any runtime required by the chosen implementation language, and the selected interface and graphics libraries. Loading source files also depends on local file access. Specific products and versions remain to be selected and recorded in the design documentation.

No external compilation service is required by the current scope. The team must distinguish tools used during development from software required on an end user’s computer. Installation instructions should identify the latter once those dependencies have been selected.

4. Decisions to be Resolved

We must establish the maximum source size and the treatment of executions that do not terminate or generate very large traces. Limits may need to address execution steps and retained states as well as source lines. No numeric limit is assigned here because the draft provides no measured or agreed threshold.

Other open decisions include the supported Java version and constructs, the treatment of runtime errors, the exact granularity of an execution step, and measurable responsiveness targets. A single source line can contain several operations, so “line-by-line” must not imply that every line always produces exactly one state.

5. Managing Changes

When an open decision is resolved, we should update this specification and the associated acceptance examples together. Changes that expand language support or introduce new interaction modes should be evaluated against the remaining milestones before they become release commitments.

7. APPORTIONING OF REQUIREMENTS

To ensure project milestones are met, requirements are apportioned into different phases:

Milestone 1

- Compare collaboration tools for source control, documents and presentations, communication, task tracking, and scheduling. Review application technologies and document the basis for the desktop implementation. Investigate strategies for Java lexical analysis, parsing, and execution modeling, with particular attention to the team's knowledge gap in compilers.
- Create and demonstrate an initial memory diagram design. The prototype should establish how primitive values, reference variables, strings, objects, and method frames will be distinguished. Prepare the requirements document, software design document, and test plan so that the later implementation has an agreed starting point.
- The milestone review should produce a documented technology direction, an initial supported-language proposal, and representative diagrams. It should also identify unresolved technical questions that must be addressed before the parser and diagram generator can be combined.

Milestone 2

- Select the tool or library used to display memory diagrams. Implement, test, and demonstrate the Java parser and a simple diagram generator using examples within the agreed language subset. Check the intermediate state information and the resulting display to distinguish incorrect interpretation from a rendering defect.
- Draft the animation behavior and investigate layouts for more complicated memory states. Examples should exercise multiple variables, object references, and method frames when those constructs are supported. Record how labels and arrows remain understandable as the displayed state increases in size.
- The milestone review should demonstrate a path from supported source input to a basic diagram and describe the remaining work needed for navigation and animation. Documentation should reflect the actual parser behavior and the diagram conventions used by the prototype.

Milestone 3

- Implement, test, and demonstrate the memory diagram animator and rendering of the agreed complex states. Integrate the parser, state interpreter, generator, animator, and desktop interface into one cohesive workflow. Verify that navigation updates the source highlighting and diagram consistently.

- Demonstrate the complete supported workflow, including code input, analysis, forward stepping, and backward stepping. Evaluate possible extensions only after the core behavior has been assessed. Extensions remain optional unless the requirements and test plan are revised to include them.
- Final evaluation should use repeatable examples and recorded expected states. The demonstration should distinguish completed requirements from remaining limitations rather than imply support for Java constructs that have not been implemented and tested.

8. FUNCTIONAL & INTERFACE REQUIREMENTS

1. Enter and Load Source

- The application shall allow the user to enter, edit, clear, and load Java source code in the code editor. The editor shall display line numbers.
- Acceptance check: Enter and edit a sample, load a source file, and clear the editor. Confirm that the displayed text and line numbers match each operation.

2. Analyze Supported Source

- The application shall perform lexical, syntax, and semantic analysis of input within the documented Java subset and provide the program representation needed to generate execution states.
- Acceptance check: Analyze valid examples covering each supported construct and compare the resulting interpretation with the expected behavior.

3. Generate Execution States

- The application shall generate an ordered trace describing the represented values, active method frames, objects, and references at each execution step. Each step shall be associated with an applicable source location.
- Acceptance check: Use an example with changing values and inspect the expected sequence of states. Confirm that the associated source locations explain the transitions.

4. Navigate Forward

- The application shall provide toolbar controls for moving forward through the available execution states.
- Acceptance check: Advance through several states and confirm that values, frames, and reference relationships match their displayed state.

5. Synchronize Code and Diagram

- When the active step changes, the application shall update the memory diagram and code highlighting to reflect that same step.
- Acceptance check: Navigate in both directions and verify that the highlighted location corresponds to the displayed state under the documented step convention.

6. Display the Memory Model

- The application shall display primitive variables as labeled rectangles containing values, references as labeled boxes with target arrows, and strings as rounded or oval nodes. It shall distinguish active frames and represented objects.
- Acceptance check: Compare the diagram for a supported example with a manually prepared expected diagram, checking names, values, targets, and frame membership. Ensure it matches memory models present in existing research (see section 4)